

PROJECT XY

System Specifications & Architecture Document

Document Abstract

This document details the functional, technical, operational, and architectural requirements for **Project XY**: a highly secure, peer-to-peer real-time video communications platform featuring end-to-end cryptographic handshake pairing, serverless automated AI transcription, and instant legal-ready PDF session compilation.

Backend Stack:	NestJS, GraphQL, WebSockets, WebRTC Signalling
Frontend Stack:	Native HTML5, CSS3, Vanilla ECMAScript 6+
AI Integration:	OpenAI Whisper / Deepgram Live Streams & GPT-4o API
Document Version:	1.0.0 (Release Specs)
Date:	May 28, 2026

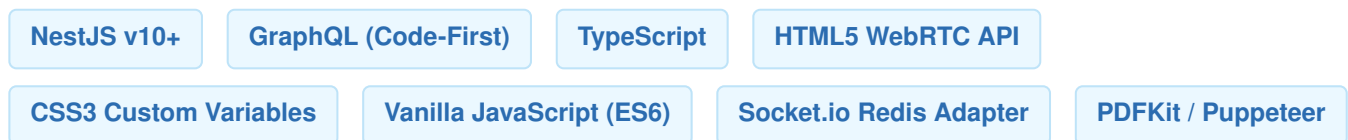
1. Executive Summary & Core Concept

Project XY is a secure, ephemeral WebRTC-driven video calling application engineered to meet contemporary demands for high-privacy digital communication. Unlike typical communication tools that rely on continuous public directories, identity profiles, or complex contact lists, Project XY operates on an ad-hoc, cryptographically gated pairing architecture.

A communication channel is established exclusively when an initiating caller generates a localized, transient cryptographic hash handshake, which must be shared securely off-band with the intended recipient. Upon verification of this handshake, an authenticated WebRTC peer-to-peer live stream commences. Concurrently, an integrated asynchronous Artificial Intelligence pipeline processes raw dual-channel audio streams into a formatted text transcript. At the absolute termination of the live call feed, both parties are instantly provisioned with a secure download compilation of the full conversation as a structured PDF document, ensuring compliance, auditing capability, and ease of reference.

2. Technical Stack Definition

To preserve scalability, ensure sub-second rendering loops, and mitigate dependency overhead, the technical architecture has been cleanly segmented into an efficient, modular ecosystem:



2.1 Backend Architecture

- **Framework:** NestJS utilizing a modular Code-First GraphQL schema execution. NestJS acts as the central router for WebRTC signaling, hash generation orchestration, and third-party AI streaming bridges.
- **API Layer:** GraphQL Queries and Mutations handle administrative tasks (e.g., initiating rooms, validating hashes, and verifying file states). GraphQL Subscriptions (backed by native WebSockets via Apollo/Socket.io) manage asynchronous live events.
- **Signaling & State Layer:** High-speed, in-memory volatile memory state management (backed optional by Redis) handling short-lived WebRTC SDP (Session Description Protocol) offers, answers, and ICE (Interactive Connectivity Establishment) candidates.

2.2 Frontend Client Layer

- **Structure & Markup:** Plain semantic HTML5 elements utilizing native APIs (e.g., `<video>`, `<audio>`, `<template>`) eliminating heavy runtime frameworks (React/Angular/Vue) to minimize browser memory footprint.

- **Styling Layer:** Pure CSS3 with structured CSS Custom Variables for uniform color systems, responsive flex/grid viewports, and clean presentation of active call feeds, transcript overlays, and input portals.
- **Application Logic:** Pure Object-Oriented Vanilla JavaScript (ES6+) managing native browser WebRTC connections (`RTCPeerConnection`), WebSocket listeners, local state rendering, and `MediaStream` processing.

3. Functional Walkthrough & System Workflow

3.1 Phase 1: Gated Room & Cryptographic Hash Generation

The user journey starts on an explicit landing view split into two distinct execution vectors: **Initiate Secure Feed** and **Join Gated Feed**.

1. The Caller clicks *"Generate Encryption Hash"*. The client triggers a GraphQL mutation `initiateSecureSession` to the NestJS cluster.
2. The backend generates a high-entropy cryptographically secure room token and couples it with a unique alphanumeric access hash sequence $H = \text{SHA-256}(\text{Salt} + \text{Timestamp} + \text{Secret})$ trimmed down to a human-readable 12-character uppercase secure key.
3. The session state is written to the backend cache with a strict Time-to-Live (TTL) of 300 seconds (5 minutes). If no recipient binds to this hash within the TTL, the room expires automatically.
4. The Caller receives the hash on their browser and presents it cleanly via the UI for copy-pasting. The system places the Caller client into a listening state, awaiting a signaling hook via a GraphQL Subscription.

3.2 Phase 2: Recipient Verification & WebRTC Handshake

1. The Recipient accesses Project XY, inputs the 12-character alphanumeric key into the validation field, and clicks *"Initialize Live Feed"*.
2. The client fires a GraphQL mutation `validateSessionHash(hash: $hash)`. The NestJS backend verifies the active existence and unlinked state of the hash inside the cache layer.
3. If valid, the room status upgrades to paired. The backend pushes an announcement down to the Caller via the open GraphQL Subscription.
4. Both clients proceed to capture local media streams via `navigator.mediaDevices.getUserMedia()` and begin standard WebRTC signaling:
 - Caller creates an SDP Offer: `peerConnection.createOffer()` and transfers it through the NestJS WebSocket signaling gateway.
 - Recipient processes the Offer, updates local state, creates an SDP Answer via `peerConnection.createAnswer()`, and passes it back.
 - Both peers continuously discover and stream ICE Candidates via the NestJS signaling bridge until a Direct P2P or TURN/STUN mediated network route binds.

5. The live video stream populates the HTML5 video tags on both clients.

3.3 Phase 3: Live Feed & Continuous AI Transcription

While video and audio payloads traverse peer-to-peer to preserve privacy and minimize latency, audio components must be cleanly transcribed by the integrated AI engine.

Architectural Choice for AI Integration

To fulfill the requirement that the text is written by an AI during the call, the system implements an **Audio Worklet Processor API** loop on the clients. This splits off low-bitrate compressed audio segments chunks every 2000ms, streaming them via the NestJS server directly into a stateful, streaming-capable Automatic Speech Recognition (ASR) service (e.g., Deepgram Live WebSockets or OpenAI Whisper Live Streaming API).

The AI service returns continuous text segments marked with time-offsets and client identification tags. The NestJS server forwards these text packets down to both clients using a GraphQL Subscription payload:

```
subscription OnTranscriptUpdated($roomId: ID!) {
  transcriptDelta(roomId: $roomId) {
    speakerId    # "Caller" or "Receiver"
    timestamp    # Unix millisecond stamp
    textSegment  # "Hello, can you hear me clearly?"
    isFinal      # Boolean indicating AI finalized the text pass
  }
}
```

The client-side Vanilla JS receives the subscription payload and dynamically updates a synchronized scrolling HTML transcript panel mapped alongside the active video viewports. The NestJS backend sequentially appends these finalized blocks into an in-memory structured log array bound to that specific `roomId`.

3.4 Phase 4: Session Termination & PDF Compilation Export

1. Either participant interacting with the *"Disconnect Feed"* button instantly destroys the local `RTCPeerConnection` and disposes of camera/mic tracking objects.
2. The frontend pushes a terminal mutation `terminateSession(roomId: $roomId)` or the signaling socket detects an abrupt disconnection event.
3. The NestJS engine updates the room state to `terminated`, flags the transcription array as finalized, and calls an isolated internal compilation module (e.g., using `pdfkit` or a headless Chrome wrapper like `puppeteer`).
4. The server compiles a beautifully structured PDF document containing: session metadata (date, exact duration, total word counts), cryptographic authentication parameters, and a chronologically mapped, speaker-attributed script layout.

5. The generated binary is saved temporarily into a localized memory buffer or a secure, short-lived storage volume, returning a transient single-use download link down to both clients.
6. The frontend UI switches views to a clean **"Session Concluded"** dashboard, presenting a prominent **"Download Conversation PDF"** button. Once downloaded or after an absolute window of 15 minutes, the PDF and transcription array are permanently wiped from the server's memory nodes, guaranteeing total ephemeral security.

4. GraphQL Schema Architecture

The system relies on a strictly defined GraphQL interface schema to govern the complex state interactions between the frontend vanilla clients and the NestJS framework.

Operation / Type	Name / Signature	Arguments	Return Type & Function
Mutation	initiateSecureSession	None	SessionKeyPair! Generates the unique room ID and the transient 12-char passcode hash for the Caller.
Mutation	validateSessionHash	hash: String!	SessionJoinResult! Invoked by the Recipient to verify credentials and unlock signaling access.
Mutation	sendSignalingPayload	roomId: ID!, payload: String!	Boolean! Routes WebRTC SDP Offers, Answers, and ICE candidates dynamically between clients.
Mutation	terminateSession	roomId: ID!	Boolean! Gracefully ends the call state and initiates the automated server-side PDF generator.
Query	pollPdfStatus	roomId: ID!	PdfStatusResult! Checks processing state and safely fetches the short-lived secure PDF download URL.
Subscription	sessionEventConnected	roomId: ID!	SessionEventPayload! Pushes immediate alerts to the Caller when a Recipient has bound to their hash.
Subscription	signalingStream	roomId: ID!	SignalingPayload! Asynchronous WebSocket stream executing raw WebRTC handshakes between the peers.
Subscription	transcriptDelta	roomId: ID!	TranscriptLine! Pushes continuous live text segments updated in real-time by the transcription AI.

5. Data Flow & Security Requirements

5.1 Cryptographic Identity Control

Project XY intentionally lacks a standard user persistency database layer. No persistent accounts, emails, or hardware identifiers are ever mapped. This prevents vector tracking or target identification attacks. Room discovery hashes must be routed through user-selected external communication networks (e.g., encrypted instant messengers or verbal relays), moving interception vulnerability completely outside the system boundaries.

5.2 WebRTC Media Security

The signaling server (NestJS) handles nothing but structural handshakes. Once the `RTCPeerConnection` transitions into an active network link, all underlying video and audio frames are encapsulated directly inside the standard **Secure Real-time Transport Protocol (SRTP)** framework. Video frames are encrypted dynamically via AES-128 or AES-256 using key negotiations executed via Datagram Transport Layer Security (DTLS).

5.3 Ephemeral Server Lifecycle

The transcription process requires raw audio components to pass through the server infrastructure to utilize high-fidelity AI LLM engines. To maintain strict data minimization rules, the following policies are strictly applied at the core engine level:

- Audio chunks sent from clients are processed purely within high-speed in-memory streams. They are never written to physical disk drives.
- The live textual logs reside solely inside volatile RAM storage, tagged strictly to the active room context.
- Upon triggering `terminateSession`, a 900-second timer initiates. Once this window passes, the server triggers a deep garbage collection cycle on the memory blocks containing that room's context, rendering post-session forensic discovery impossible.

6. UI/UX Interface Blueprint

The interface follows a clean visual language built with clean CSS structure. It maximizes screen space for video feeds and presents real-time AI transcript outputs cleanly.

Screen 1: The Initialization Dashboard

- **Left Column (The Originator):** A minimalist interactive box. Contains a prominent *"Create Secure Room"* action button. Clicking it activates an animated display revealing the 12-character high-visibility key (e.g., XY-892-F39-K) with a quick *"Copy to Clipboard"* button, alongside a pulsing loading element labeled *"Awaiting remote connection..."*.

- **Right Column (The Receiver):** A simple text field designed to receive the 12-character string. Below it is an accent-colored button labeled *"Establish Connection"*. This button remains disabled until the input conforms exactly to the required alphanumeric expression.

Screen 2: The Active Communication Workspace

- **Video Viewport Engine (Main Left View - 70% Width):** A cinematic dark-themed workspace layout displaying the incoming high-definition video track from the remote peer. A smaller, draggable overlay frame displays the user's local webcam stream in the bottom-right quadrant. The bottom border houses clean control buttons for muting microphone inputs, disabling local video capture devices, and a prominent red *"End Session"* disconnect action.
- **Live AI Transcriber Stream (Right Panel - 30% Width):** A high-contrast scrolling panel with a dedicated header reading *"Live AI Telemetry Script"*. The text feeds automatically upward as sentences are compiled, applying different background colors for speech bubbles to differentiate between the participants:
 - [11:52:10 AM] Caller: "Welcome to our secure meeting. Let's review the system contract parameters."
 - [11:52:18 AM] Recipient: "Agreed. The live AI feedback loop appears to be streaming perfectly."

Screen 3: Post-Session Offboarding Display

- Triggered instantly upon session tear-down. The workspace dims out and is replaced by an overlay notification confirming the session has terminated.
- The central interface element displays a success card stating: *"Cryptographic Channel Safely Destroyed. Document Compiled Successfully."*
- Features an actionable primary download button: **"Download Complete PDF Transcript"**. A sub-text warning reads: *"Attention: This page and its compiled records are completely ephemeral. Reloading this window or waiting 15 minutes will permanently erase this file from existence."*